

SQL Notes: Types of Keys in SQL

Practical Foundation Before Joins

1. Why Do We Need Keys in SQL?

In real databases, data is stored in multiple tables. To identify records correctly and connect tables properly, we use **keys**.

Example:

customer_name	city	phone
Rahul	Bengaluru	9999999999 1
Rahul	Mumbai	9999999999 2
Priya	Delhi	9999999999 3

Here, **customer_name** cannot uniquely identify a customer because two customers have the same name: Rahul.

So we need a unique identifier.

customer_id	customer_name	city	phone
1	Rahul	Bengaluru	9999999999 1
2	Rahul	Mumbai	9999999999 2
3	Priya	Delhi	9999999999 3

Now every customer can be uniquely identified using `customer_id`.

Key Idea

Keys help us:

- Identify records uniquely
 - Avoid duplicate records
 - Connect related tables
 - Maintain data quality
 - Build correct joins
 - Design proper databases
 - Validate source and target data
-

2. Common Dataset Used in This Lecture

customers table

customer_id	customer_name	email	phone	city
1	Rahul	rahul@gmail.com	9999999991	Bengaluru
2	Priya	priya@gmail.com	9999999992	Mumbai
3	Aman	aman@gmail.com	9999999993	Delhi

orders table

order_id	customer_id	order_date	total_amount
101	1	2026-01-05	2000
102	1	2026-01-10	1500
103	2	2026-01-12	3000

products table

product_id	sku_code	product_name	category
201	SKU-LAP-01	Laptop	Electronics
202	SKU-MOU-01	Mouse	Electronics
203	SKU-CHR-01	Chair	Furniture

order_items table

order_id	product_id	quantity	item_price
101	201	1	50000
101	202	2	700
103	203	1	4000

3. Primary Key

Definition

A **Primary Key** is a column or group of columns that uniquely identifies each row in a table.

A primary key has two important rules:

- It must be unique
- It cannot be NULL

Example

In the `customers` table, `customer_id` is the primary key.

customer_id	customer_name	email
1	Rahul	rahul@gmail.com

2	Priya	priya@gmail.com
3	Aman	aman@gmail.com

Here, each customer has a unique `customer_id`.

SQL Example

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY,  
  customer_name VARCHAR(100),  
  email VARCHAR(100),  
  phone VARCHAR(15),  
  city VARCHAR(50)  
);
```

Valid Insert

```
INSERT INTO customers  
(customer_id, customer_name, email, phone, city)  
VALUES  
(1, 'Rahul', 'rahul@gmail.com', '9999999991', 'Bengaluru');
```

Invalid Insert: Duplicate Primary Key

```
INSERT INTO customers  
(customer_id, customer_name, email, phone, city)  
VALUES  
(1, 'Rahul Sharma', 'rahul2@gmail.com', '9999999994', 'Pune');
```

This will fail because `customer_id = 1` already exists.

Invalid Insert: NULL Primary Key

```
INSERT INTO customers  
(customer_id, customer_name, email, phone, city)  
VALUES  
(NULL, 'Sneha', 'sneha@gmail.com', '9999999995', 'Pune');
```

This will fail because a primary key cannot be NULL.

Practical Example

If two customers have the same name, we should not identify them by name.

Wrong approach:

```
WHERE customer_name = 'Rahul';
```

Better approach:

```
WHERE customer_id = 1;
```

Important Point

A primary key is the main identity of a row.

4. Foreign Key

Definition

A **Foreign Key** is a column in one table that refers to the primary key or unique key of another table.

It is used to create a relationship between two tables.

Example

In the `customers` table:

`customer_id` is the primary key.

In the `orders` table:

`customer_id` is a foreign key because it refers to `customers.customer_id`.

Parent and Child Table

Concept	Meaning
Parent Table	Table that contains the primary key
Child Table	Table that contains the foreign key

In this example:

Table	Role
customers	Parent table
orders	Child table

SQL Example

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT,  
  order_date DATE,  
  total_amount DECIMAL(10,2),  
  
  CONSTRAINT fk_orders_customers  
    FOREIGN KEY (customer_id)  
    REFERENCES customers(customer_id)  
);
```

Valid Insert

```
INSERT INTO orders  
(order_id, customer_id, order_date, total_amount)  
VALUES  
(101, 1, '2026-01-05', 2000);
```

This works because `customer_id = 1` exists in the `customers` table.

Invalid Insert

```
INSERT INTO orders
(order_id, customer_id, order_date, total_amount)
VALUES
(104, 99, '2026-01-15', 2500);
```

This will fail in databases where foreign key constraints are enforced.

Reason:

`customer_id = 99` does not exist in the `customers` table.

Important Point

A foreign key prevents orphan records.

Example:

An order should not exist for a customer who does not exist.

Practical Use

Foreign keys are used for:

- Connecting tables
- Maintaining referential integrity
- Avoiding invalid child records
- Writing correct joins
- Data modeling

5. Primary Key vs Foreign Key

Point	Primary Key	Foreign Key
Purpose	Uniquely identifies a row	Connects one table to another
Duplicate values	Not allowed	Can be repeated
NULL values	Not allowed	Can be allowed depending on design

Table role	Usually in parent table	Usually in child table
Example	customers.customer_id	orders.customer_id

Example

customers table:

customer_id	customer_name
1	Rahul
2	Priya

orders table:

order_id	customer_id	total_amount
101	1	2000
102	1	1500
103	2	3000

Here:

- `customers.customer_id` is the primary key
- `orders.customer_id` is the foreign key

Join Example

```
SELECT
  c.customer_id,
  c.customer_name,
  o.order_id,
  o.total_amount
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id;
```


Output

customer_id	customer_name	order_id	total_amount
1	Rahul	101	2000
1	Rahul	102	1500
2	Priya	103	3000

Key Learning

We join tables using keys because keys define relationships between tables.

6. Candidate Key

Definition

A **Candidate Key** is any column or combination of columns that can uniquely identify a row.

A table can have multiple candidate keys.

From all candidate keys, one key is selected as the primary key.

Example

customers table:

customer_id	customer_name	email	phone
1	Rahul	rahul@gmail.com	9999999991
2	Priya	priya@gmail.com	9999999992
3	Aman	aman@gmail.com	9999999993

Possible candidate keys:

- `customer_id`
- `email`
- `phone`

All three can uniquely identify a customer.

Why `customer_id` is usually selected as Primary Key

`email` can change.

`phone` can change.

But `customer_id` should remain stable.

So `customer_id` is usually a better primary key.

Important Point

All primary keys are candidate keys, but all candidate keys are not primary keys.

7. Alternate Key

Definition

An **Alternate Key** is a candidate key that was not selected as the primary key.

Example

Candidate keys in customers table:

- `customer_id`
- `email`
- `phone`

If `customer_id` is selected as the primary key, then:

- email
- phone

become alternate keys.

SQL Example

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    email VARCHAR(100) UNIQUE,  
    phone VARCHAR(15) UNIQUE,  
    city VARCHAR(50)  
);
```

Here:

- customer_id is primary key
- email is alternate key
- phone is alternate key

Practical Use

Alternate keys are useful when business columns should also remain unique.

Example:

Two customers should not register with the same email.

8. Unique Key

Definition

A **Unique Key** ensures that values in a column do not repeat.

A table can have multiple unique keys.

Example

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY,  
  customer_name VARCHAR(100),  
  email VARCHAR(100) UNIQUE,  
  phone VARCHAR(15) UNIQUE,  
  city VARCHAR(50)  
);
```

Here:

- `customer_id` is primary key
- `email` is unique
- `phone` is unique

Valid Insert

```
INSERT INTO customers  
(customer_id, customer_name, email, phone, city)  
VALUES  
(1, 'Rahul', 'rahul@gmail.com', '9999999991', 'Bengaluru');
```

Invalid Insert: Duplicate Email

```
INSERT INTO customers  
(customer_id, customer_name, email, phone, city)  
VALUES  
(2, 'Rahul Sharma', 'rahul@gmail.com', '9999999992', 'Mumbai');
```

This will fail because `email = 'rahul@gmail.com'` already exists.

Primary Key vs Unique Key

Point	Primary Key	Unique Key
Main purpose	Main identity of row	Prevent duplicate values
NULL allowed?	No	Depends on database
Count per table	One primary key constraint	Multiple unique keys possible
Example	customer_id	email, phone

Important Point

A primary key is always unique, but a unique key is not always the primary key.

9. Super Key

Definition

A **Super Key** is any column or combination of columns that can uniquely identify a row.

A super key may contain extra columns.

Example

customers table:

customer_id	customer_name	email	city
1	Rahul	rahul@gmail.com	Bengaluru
2	Priya	priya@gmail.com	Mumbai

Possible super keys:

- `customer_id`
- `email`
- `customer_id + customer_name`
- `customer_id + city`
- `email + city`

Explanation

`customer_id` alone is enough to identify the row.

But `customer_id + city` also identifies the row.

However, `city` is extra and unnecessary.

So `customer_id + city` is a super key, but not a candidate key.

Super Key vs Candidate Key

Concept	Meaning
Super Key	Any column or combination that uniquely identifies a row
Candidate Key	Minimal super key with no unnecessary columns

10. Composite Key

Definition

A **Composite Key** is a key made using two or more columns.

It is used when one column alone cannot uniquely identify a row.

Example

order_items table:

order_id	product_id	quantity	item_price
101	201	1	50000
101	202	2	700
103	203	1	4000

Here:

`order_id` alone is not unique.

`product_id` alone is not unique.

But the combination of `order_id + product_id` is unique.

So `order_id + product_id` can be a composite key.

SQL Example

```
CREATE TABLE order_items (  
  order_id INT,  
  product_id INT,  
  quantity INT,  
  item_price DECIMAL(10,2),  
  
  PRIMARY KEY (order_id, product_id)  
);
```

Valid Insert

```
INSERT INTO order_items  
(order_id, product_id, quantity, item_price)  
VALUES  
(101, 201, 1, 50000);
```

Another Valid Insert

```
INSERT INTO order_items  
(order_id, product_id, quantity, item_price)  
VALUES  
(101, 202, 2, 700);
```

This works because the combination is different.

Invalid Insert

```
INSERT INTO order_items  
(order_id, product_id, quantity, item_price)  
VALUES  
(101, 201, 3, 50000);
```

This fails because the combination of `order_id = 101` and `product_id = 201` already exists.

Practical Use Cases

Composite keys are common in:

- Order item tables
 - Student-course enrollment tables
 - Employee-project mapping tables
 - User-role mapping tables
 - Fact tables with multiple dimensions
-

11. Composite Foreign Key

Definition

A **Composite Foreign Key** is a foreign key made using two or more columns.

It references a composite primary key in another table.

Example

course_enrollments table:

```
CREATE TABLE course_enrollments (  
  student_id INT,  
  course_id INT,  
  enrollment_date DATE,  
  
  PRIMARY KEY (student_id, course_id)  
);
```

assessment_scores table:

```
CREATE TABLE assessment_scores (  
  student_id INT,  
  course_id INT,  
  assessment_name VARCHAR(100),  
  score INT,  
  
  FOREIGN KEY (student_id, course_id)  
    REFERENCES course_enrollments(student_id, course_id)  
);
```


Explanation

Here, `assessment_scores` references the exact enrollment of a student in a course.

So both `student_id` and `course_id` are needed.

This is a composite foreign key.

12. Natural Key

Definition

A **Natural Key** is a real-world business column that can uniquely identify a record.

Examples

Entity	Natural Key
Customer	Email or phone
Product	SKU code
Employee	Employee code
Vehicle	Registration number
Citizen	Aadhaar number

Example

```
CREATE TABLE products (  
  product_id INT PRIMARY KEY,  
  sku_code VARCHAR(50) UNIQUE,  
  product_name VARCHAR(100),  
  category VARCHAR(50)  
);
```

Here, `sku_code` is a natural key because it comes from the business.

Important Point

Natural keys have business meaning, but they can change.

Examples:

- Customer email can change
- Phone number can change
- Product SKU can change during migration
- Employee code format can change

So in many systems, natural keys are stored as unique columns, but not used as the main primary key.

13. Surrogate Key

Definition

A **Surrogate Key** is an artificial key generated by the system.

It has no business meaning.

It is mainly used to uniquely identify records.

Example

```
CREATE TABLE customers (  
  customer_id INT AUTO_INCREMENT PRIMARY KEY,  
  customer_name VARCHAR(100),  
  email VARCHAR(100) UNIQUE,  
  phone VARCHAR(15) UNIQUE,  
  city VARCHAR(50)  
);
```

Here, `customer_id` is a surrogate key.

It is generated by the system.

Example Data

customer_id	customer_name	email
1	Rahul	rahul@gmail.com
2	Priya	priya@gmail.com
3	Aman	aman@gmail.com

The values 1, 2, and 3 do not have business meaning.

They are generated only to identify rows.

Natural Key vs Surrogate Key

Point	Natural Key	Surrogate Key
Meaning	Has business meaning	No business meaning
Example	email, phone, SKU	customer_id, product_id
Can change?	Yes, sometimes	Should not change
Generated by	Business/system source	Database/system
Common in warehouse?	Stored as business key	Used as dimension key

Practical Data Engineering Point

In data warehousing, surrogate keys are heavily used in dimension tables.

Example:

`customer_sk` can be used as a surrogate key in a customer dimension table.

This is useful when customer information changes over time.

14. Secondary Key

Definition

A **Secondary Key** usually means a column used for searching, filtering, or indexing.

It may not uniquely identify a row.

Example

customers table:

customer_id	customer_name	email	city
1	Rahul	rahul@gmail.com	Bengaluru
2	Priya	priya@gmail.com	Mumbai
3	Aman	aman@gmail.com	Delhi

Here, **customer_id** is the primary key.

But users may frequently search customers by city.

```
SELECT
  customer_id,
  customer_name,
  city
FROM customers
WHERE city = 'Bengaluru';
```

So we can create an index on city.

```
CREATE INDEX idx_customers_city
ON customers(city);
```

Important Point

A secondary key helps with searching and performance.

It does not always uniquely identify a row.

15. NOT NULL Constraint vs Key

Definition

NOT NULL is not a key.

It is a constraint that makes sure a column must have a value.

Example

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY,  
  customer_name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  city VARCHAR(50)  
);
```

Here:

- `customer_id` is primary key
- `customer_name` is mandatory
- `email` is unique and mandatory

Difference

Constraint	Meaning
NOT NULL	Value is required
UNIQUE	Value cannot repeat
PRIMARY KEY	Value is required and cannot repeat
FOREIGN KEY	Value should refer to another table

16. Practical Ecommerce Table Design

customers table

```
CREATE TABLE customers (  
  customer_id INT AUTO_INCREMENT PRIMARY KEY,  
  customer_name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  phone VARCHAR(15) UNIQUE,  
  city VARCHAR(50)  
);
```

products table

```
CREATE TABLE products (  
  product_id INT AUTO_INCREMENT PRIMARY KEY,  
  sku_code VARCHAR(50) UNIQUE NOT NULL,  
  product_name VARCHAR(100) NOT NULL,  
  category VARCHAR(50),  
  price DECIMAL(10,2)  
);
```

orders table

```
CREATE TABLE orders (  
  order_id INT AUTO_INCREMENT PRIMARY KEY,  
  customer_id INT NOT NULL,  
  order_date DATE,  
  total_amount DECIMAL(10,2),  
  
  CONSTRAINT fk_orders_customers  
    FOREIGN KEY (customer_id)  
    REFERENCES customers(customer_id)  
);
```

order_items table

```
CREATE TABLE order_items (  
  order_id INT,  
  product_id INT,
```

```
quantity INT,  
item_price DECIMAL(10,2),  
  
PRIMARY KEY (order_id, product_id),  
  
CONSTRAINT fk_order_items_orders  
  FOREIGN KEY (order_id)  
  REFERENCES orders(order_id),  
  
CONSTRAINT fk_order_items_products  
  FOREIGN KEY (product_id)  
  REFERENCES products(product_id)  
);
```

Explanation

This is a proper relational design.

- `customers.customer_id` uniquely identifies a customer
 - `products.product_id` uniquely identifies a product
 - `orders.order_id` uniquely identifies an order
 - `orders.customer_id` connects orders with customers
 - `order_items.order_id` connects order items with orders
 - `order_items.product_id` connects order items with products
 - `order_items` uses a composite primary key: `order_id + product_id`
-

17. How Keys Help in Joins

Example 1: Customer and Orders

```
SELECT  
  c.customer_name,  
  o.order_id,  
  o.total_amount  
FROM customers c  
JOIN orders o  
  ON c.customer_id = o.customer_id;
```

Here:

- `customers.customer_id` is primary key
- `orders.customer_id` is foreign key

Example 2: Products and Order Items

```
SELECT
  oi.order_id,
  p.product_name,
  oi.quantity,
  oi.item_price
FROM order_items oi
JOIN products p
  ON oi.product_id = p.product_id;
```

Here:

- `products.product_id` is primary key
- `order_items.product_id` is foreign key

Example 3: Complete Order Details

```
SELECT
  c.customer_name,
  o.order_id,
  p.product_name,
  oi.quantity,
  oi.item_price,
  oi.quantity * oi.item_price AS item_total
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id
JOIN order_items oi
  ON o.order_id = oi.order_id
JOIN products p
  ON oi.product_id = p.product_id;
```

Key Learning

Joins become clear when keys are clear.

Without keys, table relationships become confusing.

With keys, we know exactly how tables are connected.

18. Summary of Key Types

Key Type	Definition	Example
Primary Key	Uniquely identifies each row	customers.customer_id
Foreign Key	Refers to key of another table	orders.customer_id
Candidate Key	Possible column that can uniquely identify row	customer_id, email, phone
Alternate Key	Candidate key not selected as primary key	email, phone
Unique Key	Prevents duplicate values	email UNIQUE
Super Key	Any unique identifying column combination	customer_id + city
Composite Key	Key made of two or more columns	order_id + product_id
Composite Foreign Key	Multi-column foreign key	student_id + course_id
Natural Key	Real-world business identifier	email, SKU, employee_code
Surrogate Key	System-generated identifier	customer_id, product_id
Secondary Key	Used for searching/indexing	city index

19. Interview Questions Based on This Lecture

Question 1

You have a customers table where two customers can have the same name. Which column should be used to uniquely identify a customer?

Answer

We should use a unique identifier such as `customer_id`.

`customer_name` should not be used because names can repeat.

`customer_id` is a better choice because it uniquely identifies each customer.

Question 2

In an ecommerce system, every order should belong to a valid customer. Which key relationship will help enforce this?

Answer

A foreign key relationship should be used.

`customers.customer_id` should be the primary key.

`orders.customer_id` should be the foreign key referencing `customers.customer_id`.

This ensures that an order cannot be created for a non-existing customer.

Question 3

A customer can place multiple orders. Should `orders.customer_id` be unique?

Answer

No.

`orders.customer_id` should not be unique because one customer can place many orders.

The same `customer_id` can appear multiple times in the orders table.

But `orders.customer_id` should be a foreign key referencing `customers.customer_id`.

Question 4

Can `order_id` be the primary key of the orders table?

Answer

Yes.

`order_id` can be the primary key because each order should have a unique order ID.

It should not repeat and should not be NULL.

Question 5

In the customers table, `customer_id`, `email`, and `phone` are all unique. What are these called?

Answer

They are candidate keys because each of them can uniquely identify a customer.

If we choose `customer_id` as the primary key, then `email` and `phone` become alternate keys.

Question 6

Why is `customer_id` usually better than `email` as a primary key?

Answer

Because email can change.

A customer may update their email address in the future.

But `customer_id` should remain stable.

So `customer_id` is usually a better primary key.

Question 7

In an `order_items` table, one order can have multiple products. Can `order_id` alone be the primary key?

Answer

No.

`order_id` alone cannot be the primary key because the same order can have multiple products.

A better primary key can be the combination of `order_id` and `product_id`.

This is called a composite key.

Question 8

What is the composite key in this table?

<code>order_id</code>	<code>product_id</code>	<code>quantity</code>
101	201	1
101	202	2
103	203	1

Answer

The composite key is:

`order_id + product_id`

Together, they uniquely identify each row.

Question 9

A products table has `product_id` and `sku_code`. `product_id` is generated by the system, while `sku_code` comes from the business. Which one is surrogate key and which one is natural key?

Answer

`product_id` is a surrogate key because it is generated by the system and has no business meaning.

`sku_code` is a natural key because it has business meaning and identifies a product in the real world.

Question 10

Why do we use foreign keys in relational databases?

Answer

Foreign keys are used to maintain relationships between tables.

They ensure that child table records point to valid parent table records.

They help avoid orphan records.

Example:

An order should not exist for a customer who does not exist.

Question 11

What is the difference between a primary key and a unique key?

Answer

A primary key is the main identity of a table. It cannot be NULL and cannot be duplicate.

A unique key also prevents duplicate values, but it is not necessarily the main identity of the table.

A table usually has one primary key but can have multiple unique keys.

Question 12

Can a table have more than one primary key?

Answer

A table can have only one primary key constraint.

But that primary key can contain multiple columns.

When a primary key uses multiple columns, it is called a composite primary key.

Question 13

A company frequently searches customers by city. Should city be a primary key?

Answer

No.

City should not be a primary key because many customers can belong to the same city.

But we can create an index on city to improve search performance.

This can be considered a secondary key or secondary index use case.

Question 14

What is a super key?

Answer

A super key is any column or combination of columns that can uniquely identify a row.

It may contain extra columns.

Example:

If `customer_id` alone can identify a customer, then `customer_id + city` is also a super key, but it has an unnecessary extra column.

Question 15

What is the difference between a candidate key and a super key?

Answer

A super key can have extra columns.

A candidate key is a minimal super key.

Example:

`customer_id` is a candidate key.

`customer_id + city` is a super key but not a candidate key because city is unnecessary.

Question 16

You are designing a student-course enrollment table. One student can enroll in multiple courses, and one course can have multiple students. What can be the primary key?

Answer

The primary key can be a composite key:

`student_id + course_id`

Because one column alone is not enough to uniquely identify an enrollment.

Question 17

Why are keys important before learning joins?

Answer

Joins are based on relationships between tables.

Keys define those relationships.

For example:

`customers.customer_id` is the primary key.

`orders.customer_id` is the foreign key.

So we join them using:

ON `customers.customer_id` = `orders.customer_id`

If keys are clear, joins become much easier to understand.

Question 18

In a data warehouse, why do we often use surrogate keys in dimension tables?

Answer

Surrogate keys are stable system-generated identifiers.

They are useful because business keys like email, phone, or SKU may change over time.

In dimension tables, surrogate keys help track records cleanly, especially in Slowly Changing Dimension scenarios.

Question 19

What is an orphan record?

Answer

An orphan record is a child table record that does not have a matching parent table record.

Example:

An order with `customer_id = 99`, where customer 99 does not exist in the customers table.

Foreign keys help prevent orphan records.

Question 20

A customers table has `email UNIQUE NOT NULL`. Is email a primary key?

Answer

Not necessarily.

Email is unique and mandatory, so it can be a candidate key.

But if `customer_id` is selected as the primary key, then email becomes an alternate key.

20. Final Revision Points

- Keys help identify records and connect tables.
- Primary key uniquely identifies each row.
- Foreign key connects one table with another table.
- Candidate key is a possible primary key.
- Alternate key is a candidate key not selected as primary key.
- Unique key prevents duplicate values.
- Super key is any unique identifying combination.
- Candidate key is a minimal super key.
- Composite key uses multiple columns.
- Natural key has business meaning.
- Surrogate key is system-generated.
- Secondary key is used for searching or indexing.

- Keys are the foundation of joins.